

A Formula Dictionary for the HAETAE NTT Verification in EasyCrypt

Codex Proof Note

22 April 2026

Abstract

This note organizes the principal formulas underlying the HAETAE NTT verification project in EasyCrypt. Its purpose is not to restate the current proof status, but rather to index the development by mathematical task: for each task in the project, we record the central identities, invariants, representation maps, and target equivalences that must be established. The result is a formula-oriented guide to the project, written in the style of a mathematical aide-memoire for cryptographic verification.

1 Global Verification Objective

Let p be a polynomial represented concretely by a byte array $\text{rp} : \text{BArray1024.t}$. The final end-to-end objective of the project is the pair of Hoare-style statements

$$\{\text{poly_repr}(\text{rp}, p)\} \text{poly_ntt_jazz}(\text{rp}) \{\text{poly_repr}(\text{res}, \text{Rq.ntt}(p))\},$$
$$\{\text{poly_repr}(\text{rp}, p)\} \text{poly_invntt_jazz}(\text{rp}) \{\text{poly_repr}(\text{res}, \text{Rq.invntt}(p))\}.$$

Every other formula in the project serves this final pair of correctness theorems.

2 Ambient Notation

The field is $\mathbf{F}_q$ with

$$q = 64513.$$

The transform length is

$$N = 256,$$

and the chosen primitive 512-th root of unity is

$$zroot = 426 \in \mathbf{F}_q.$$

The Montgomery constant used throughout the extracted arithmetic is

$$R = 2^{32} \bmod q = 14321.$$

The bit-reversal permutation on 8 bits is written

$$\text{br}(i) = \text{bsrev}_8(i).$$

3 Task I: Imperative NTT Specifications

The forward imperative specification `NTT.ntt` executes Cooley–Tukey butterflies with layer lengths

$$128, 64, 32, 16, 8, 4, 2, 1.$$

The inverse imperative specification `NTT.invntt` executes inverse layers with lengths

$$1, 2, 4, 8, 16, 32, 64, 128,$$

followed by a global scaling by `inv(256)`.

The forward butterfly is

$$t = \zeta \cdot r_{j+\ell}, \quad r'_{j+\ell} = r_j - t, \quad r'_j = r_j + t.$$

The inverse butterfly is

$$t = r_j, \quad r'_j = t + r_{j+\ell}, \quad r'_{j+\ell} = \zeta \cdot (t - r_{j+\ell}).$$

The formal tasks attached to these formulas are:

- prove losslessness of `NTT.ntt`;
- prove losslessness of `NTT.invntt`;
- use these imperative models as refinement targets for the extracted code.

4 Task II: Termination Variants

For the forward procedure the variants are:

$$V_{\text{outer}} = \text{len}, \quad V_{\text{middle}} = 256 - \text{start}, \quad V_{\text{inner}} = \text{start} + \text{len} - j.$$

For the inverse procedure one uses:

$$V_{\text{outer}} = 256 - \text{len}, \quad V_{\text{middle}} = 256 - \text{start}, \quad V_{\text{inner}} = \text{start} + \text{len} - j,$$

and for the final scaling loop

$$V_{\text{scale}} = 256 - j.$$

These formulas belong to the purely structural task of proving `islossless`. They do not encode semantics of the transform itself, but they are indispensable for any subsequent program logic argument.

5 Task III: Root-of-Unity Structure

The transform depends on the order of `zroot`. The essential identities are

$$zroot^{128} = 28837, \quad zroot^{256} = -1, \quad zroot^{512} = 1.$$

In EasyCrypt, the decisive lemma is

$$\text{Zq.exp zroot 256} = \text{incoeff } (-1).$$

This task supports:

- the correctness of the inverse transform;
- the algebraic interpretation of the twiddle tables;
- the final normalization by `inv(256)`.

6 Task IV: Abstract NTT Formulas

The abstract forward transform in `Rq.ec` is

$$\text{ntt}(p)[i] = \begin{cases} \sum_{j=0}^{127} p_{2j} \text{zroot}^{(2 \text{br}(i/2)+1)j}, & i \equiv 0 \pmod{2}, \\ \sum_{j=0}^{127} p_{2j+1} \text{zroot}^{(2 \text{br}(i/2)+1)j}, & i \equiv 1 \pmod{2}. \end{cases}$$

The abstract inverse transform is

$$\text{invntt}(p)[i] = \begin{cases} \sum_{j=0}^{127} \text{inv}(128) p_{2j} \text{zroot}^{-(2 \text{br}(j)+1)(i/2)}, & i \equiv 0 \pmod{2}, \\ \sum_{j=0}^{127} \text{inv}(128) p_{2j+1} \text{zroot}^{-(2 \text{br}(j)+1)(i/2)}, & i \equiv 1 \pmod{2}. \end{cases}$$

This is the target mathematical semantics to which the imperative routines must eventually be shown equal.

7 Task V: Representation of Concrete Program State

The concrete-to-abstract bridge is given by

$$\text{word_to_coeff}(w) = \text{incoeff}(\text{to_sint}(w)),$$

$$\text{barray256_to_poly}(\text{rp})[i] = \text{word_to_coeff}(\text{BArray1024.get32 rp } i),$$

and

$$\text{poly_repr}(\text{rp}, p) :\Longleftrightarrow \text{barray256_to_poly}(\text{rp}) = p.$$

The formulas attached to this task are:

- pointwise readback:

$$\text{poly_repr}(\text{rp}, p) \Longrightarrow p[i] = \text{word_to_coeff}(\text{BArray1024.get32 rp } i);$$

- small signed-word conversion:

$$|z| < q \Longrightarrow \text{word_to_coeff}(\text{W32.of_int } z) = \text{incoeff}(z);$$

- normalized representative conversion:

$$|z| < q \Longrightarrow \text{word_to_coeff}(\text{W32.of_int } z) = \text{incoeff}(\text{if } 0 \leq z \text{ then } z \text{ else } z + q).$$

This task is the basic language in which all simulation statements are written.

8 Task VI: Twiddle Tables

8.1 Forward Table

The abstract forward twiddle table is

$$\zeta_k = zroot^{\text{br}(k)}.$$

Its Montgomery form is

$$\zeta_k^{\text{Mont}} = \zeta_k \cdot R.$$

The concrete table `jzetas` stores precisely these Montgomery values, except that index 0 is padding:

$$\text{jzetas}[0] = 0, \quad \text{jzetas}[k] \rightsquigarrow \zeta_k \cdot R \quad (1 \leq k \leq 255).$$

Thus the intended bridge formula is

$$\text{barray256_to_poly}(\text{jzetas}) = (\text{array256_mont}(\zeta))[0 \leftarrow 0].$$

8.2 Inverse Table

The abstract inverse twiddle table is

$$\zeta_k^{-1} = zroot^{-(\text{br}(k)+1)} \quad (0 \leq k \leq 254),$$

while the last slot is the inverse normalization:

$$\zeta_{255}^{-1} = \text{inv}(256).$$

Its Montgomery form is

$$(\zeta_k^{-1})^{\text{Mont}} = \zeta_k^{-1} \cdot R.$$

The concrete table `jzetas_inv` matches this at indices

$$0 \leq k \leq 254,$$

but at index 255 it stores the machine value

$$-29720,$$

which encodes the final scaling constant in the format expected by the Montgomery multiplication routine.

Accordingly, the bridge formula is

$$\text{barray256_to_poly}(\text{jzetas_inv}) = (\text{array256_mont_inv}(\zeta^{-1}))[255 \leftarrow -29720].$$

Remark 1. *This is the most delicate formula in the current project state. It is not conceptually difficult, but it sits precisely at the interface between the mathematics of twiddle factors and the byte-level normalization performed by the EasyCrypt libraries.*

9 Task VII: Inverse Scaling Constant

Two related constants appear in the inverse transform:

$$\text{scale255} = \text{inv}(256) = -252 \pmod{q},$$

and the machine value

$$f = -29720.$$

The key identity is the Montgomery compatibility relation

$$\text{incoeff}(f) \cdot R^{-1} = \text{scale255} \cdot R.$$

This formula serves the final loop of the inverse extracted code, where the program multiplies each coefficient by the machine word stored at `jzetas_inv[255]` via the Montgomery multiplication routine.

10 Task VIII: Montgomery Multiplication

The extracted routine `M.__fqmul` computes

$$\text{MontMul}(a, b) = a \cdot b \cdot R^{-1} \pmod{q},$$

with the result again in Montgomery representation when both inputs are already Montgomery-encoded.

The intended correctness formula is

$$\text{word_to_coeff}(\text{M.__fqmul}(a, b)) = \text{word_to_coeff}(a) \cdot \text{word_to_coeff}(b) \cdot R.$$

This task depends on the Montgomery-reduction theorem from `Montgomery.ec` and supplies the arithmetic core of every butterfly simulation.

11 Task IX: Forward Butterfly Simulation

Let the concrete word read from the table represent $\zeta \cdot R$. Let the current concrete coefficients represent x and y . Then the extracted forward butterfly computes, at the field level,

$$t = \zeta y, \quad x' = x + t, \quad y' = x - t.$$

Thus the local proof obligation is:

$$\text{concrete forward butterfly} \implies \text{imperative forward butterfly in } \text{NTT.ntt}.$$

This task combines the formulas from:

- the representation map;
- the forward twiddle bridge;
- Montgomery multiplication correctness;
- field addition and subtraction correctness.

12 Task X: Inverse Butterfly Simulation

For the inverse transform one must prove the corresponding local statement:

$$t = x, \quad x' = x + y, \quad y' = \zeta(x - y).$$

At the program level the order of updates matters, so the precise proof must track the concrete sequence:

$$t \leftarrow r_j, \quad r_j \leftarrow t + r_{j+\ell}, \quad r_{j+\ell} \leftarrow t - r_{j+\ell}, \quad r_{j+\ell} \leftarrow \zeta \cdot r_{j+\ell}.$$

This task is the inverse analogue of the previous section, augmented by the final scaling formula from Task VII.

13 Task XI: Loop Invariants for the Extracted Jasmin Code

The forward and inverse extracted procedures each contain three nested loops. The invariants must express both control-state facts and semantic progress.

Typical outer-loop ingredients are:

$$0 < \text{len} \leq 128, \quad \text{len} = 2^\ell, \quad \text{poly_repr}(\text{rp}, r),$$

together with the statement that all fully completed stages of the transform already agree on both sides.

Typical inner-loop ingredients are:

$$\text{start} \leq j \leq \text{start} + \text{len},$$

and equality of all coefficients outside the active butterfly window.

This task turns the local butterfly formulas into full procedure-level refinement theorems.

14 Task XII: Imperative-to-Abstract Identification

After the extracted code has been related to the imperative model, one must still prove the mathematical identities

$$\text{NTT.ntt}(p, \zeta) = \text{Rq.ntt}(p), \quad \text{NTT.invntt}(p, \zeta^{-1}) = \text{Rq.invntt}(p).$$

The formulas governing this task are not low-level machine identities, but rather structural equalities between:

- staged butterfly decompositions;
- the bit-reversal exponents $2 \text{br}(i) + 1$;
- the closed-form sums in `Rq.ec`.

This is where the Cooley–Tukey factorization must be matched to the explicit ring-theoretic definition.

15 Task XIII: Final Composition

At the end of the project the formulas from all prior tasks are composed:

$$\text{concrete state} \xrightarrow{\text{poly_repr}} \text{abstract polynomial} \xrightarrow{\text{Rq.ntt or Rq.invntt}} \text{target polynomial},$$

while the program side follows

$$\text{poly_ntt_jazz} \quad \text{or} \quad \text{poly_invntt_jazz}.$$

The completed chain should read:

$$\text{poly_ntt_jazz} \sim \text{M.}_\text{poly_ntt} \sim \text{NTT.ntt} = \text{Rq.ntt},$$

and similarly

$$\text{poly_invntt_jazz} \sim \text{M.}_\text{poly_invntt} \sim \text{NTT.invntt} = \text{Rq.invntt}.$$

16 Task-to-Formula Table

Task	Governing formula	Role in the proof
Imperative specs	forward and inverse butterfly recurrences	semantic target for extracted-code refinement
Losslessness	explicit integer variants for all loops	well-formedness of Hoare and pRHL reasoning
Root structure	$zroot^{256} = -1, zroot^{512} = 1$	algebraic foundation for inverse correctness
Representation	$\text{poly_repr}(\text{rp}, p)$	concrete-to-abstract state relation
Forward zetas	$\text{jzetas}[k] \rightsquigarrow zroot^{\text{br}(k)} R$	table-read interpretation in forward butterflies
Inverse zetas	$\text{jzetas_inv}[k] \rightsquigarrow zroot^{-(\text{br}(k)+1)} R$	table-read interpretation in inverse butterflies
Inverse scale	$\text{incoeff}(-29720)R^{-1}$ $\text{inv}(256)R$	= final inverse normalization
Montgomery multiply	$a \cdot b \cdot R^{-1}$ modulo q	arithmetic core of extracted multiplication
Forward local step	$(x, y) \mapsto (x + \zeta y, x - \zeta y)$	atomic forward simulation
Inverse local step	$(x, y) \mapsto (x + y, \zeta(x - y))$	atomic inverse simulation
Loop refinement	invariants $\text{len}, \text{start}, \text{j}, \text{zetasctr}$	on global simulation of extracted routines
Abstract identification	imperative staged transform = sum formula in Rq.ec	connection to mathematical specification
Final theorem	$\{\text{poly_repr}\} P \{\text{poly_repr} \circ \text{Rq.ntt}\}$	end-to-end program correctness

Conclusion

The project admits a clean mathematical decomposition. Each task is governed by a small family of formulas: arithmetic identities for roots of unity and Montgomery constants; representation formulas linking words to coefficients; local butterfly recurrences; global loop invariants; and finally the closed transform formulas from the ring-theoretic specification. Writing the project in this task-by-formula form makes its architecture transparent: the entire verification is the controlled composition of these formulas inside the proof language of EasyCrypt.